TE 4

Post-traitement d'images binaires - Etiquetage en composantes connexes

Remarques importantes :

— Consacrez du temps aux questions théoriques et/ou d'analyses. Elles sont aussi importantes que le codage !
— Consultez la correction dès qu'elle sera mise en ligne

— Vous trouverez de l'aide en ligne aux adresses suivantes :
  — OpenCV : `http://docs.opencv.org/3.2.0/`
  — Pour un accès à l'aide en ligne sur les fonctions OpenCV en python : onglet "Sphinx documentation" à ce niveau d'arborescence
  — Tutoriels OpenCV-Python : cliquer sur l'item OpenCV-Python tutorials dans la liste des modules OpenCV
  — **Pour ce TE :**
    `https://docs.opencv.org/3.2.0/d9/d61/tutorial_py_morphological_ops.html`
    `https://docs.opencv.org/3.2.0/d4/d73/tutorial_py_contours_begin.html`
  — Numpy,Scipy : `http://www.numpy.org/` et `https://docs.scipy.org/doc/`
  — Matplotlib : `http://matplotlib.org/`

Ouvrir systématiquement un onglet vers la documentation OpenCV (cf liens ci-dessus) pour rechercher toutes les informations nécessaires sur une fonction (paramètres d'entrée, de sortie, fonctionnalité, ...) et un autre sur le(s) tutoriel(s) associé(s) au TE

1 Post-traitement d'une image binarisée : opérateurs de morphologie mathématique

L'objectif de cette partie est d'évaluer les effets d'une érosion, d'une dilatation, d'une ouverture et d'une fermeture sur une image binaire.
Les fonctions utiles sont : `cv2.erode() - cv2.dilate()`
Pour la 1ère question, on considèrera un élément structurant rectangulaire de taille 3*3 (défini potentiellement à l'aide de la fonction `cv2.getStructuringElement`).

1. Ecrire un programme qui :
   (a) binarise une image (par exemple *code_postal.png* ou *voilier_oies_blanches.jpg* convertie au préalable en niveaux de gris)
   (b) érode l'image binaire obtenue et affiche l'image érodée
   (c) dilate l'image binaire obtenue à la question 1 et affiche l'image dilatée
   (d) applique une ouverture à l'image binaire obtenue en 1 et affiche le résultat
   (e) applique une fermeture à l'image binaire obtenue en 1 et affiche le résultat
   (f) applique un opening-closing (ouverture suivie d'une fermeture) à l'image binaire obtenue en 1 et affiche le résultat

2. Interpréter les résultats : les résultats observés sont-ils ceux attendus et pourquoi ? Assurez-vous que vous avez bien compris le principe d'une érosion et d'une dilatation

3. Que se passe-t-il si on change l'élément structurant ? Faites-en l'expérience !

4. Implémenter les traitements précédents à l'aide de la fonction `cv2.morphologyEx`

2 Extraction et affichage des composantes connexes

Vous avez vu en cours qu'une segmentation d'images par seuillage de l'histogramme comportait deux phases : une phase de seuillage de l'histogramme (par ex, une binarisation) et une phase dite d'étiquetage en composantes connexes.

1. Binariser l'image *voilier_oies_blanches.jpg*, convertie au préalable en niveaux de gris, en deux classes : "fond" (pixels à 0) et "oiseau" (pixels à 1 ou 255)

2. Etiqueter en composantes connexes l'image binaire obtenue à la question précédente, par application de la fonction `cv2.connectedComponents` ou `label` (fonction de `scipy.ndimage`) et visualiser le résultat

3. Analyser les résultats obtenus : le nombre de composantes connexes identifiées vous parait-il correct ? Pourquoi ?

4. Alternative aux fonctions utilisées à la question 2 :
   (a) Prendre connaissance des paramètres de la fonction `cv2.findContours` et décrire en quelques mots ce que cette fonction réalise exactement (aide en ligne + `https://docs.opencv.org/3.2.0/d4/d73/tutorial_py_contours_begin.html`)
   (b) Par application de cette fonction à l'image binaire obtenue en 1, extraire les contours externes et internes des composantes connexes
   (c) Incruster les contours extraits dans l'image couleur *voilier_oies_blanches.jpg* à l'aide de la fonction `cv2.drawContours` et visualiser l'image résultante :
   — afficher par exemple les contours externes en jaune
   — rajouter une contrainte à l'affichage : l'intérieur des composantes connexes sera visualisé en cyan
   (d) Adapter le code afin qu'il ne s'applique plus à l'image *voilier_oies_blanches.jpg* mais à un flux vidéo acquis par Webcam et converti en niveaux de gris (orienter la Webcam vers un objet (un crayon, un cube de couleur, ...) qui repose sur un fond le plus uni possible)
   Important : Pour ceux qui n'ont pas de Webcam, vous considérerez l'un des deux fichiers .avi disponibles sous les données utiles du TE3 (nom : Film AVI n°1 pour ceux qui n'ont pas Webcam, ou Film AVI n°2 ...)
   (e) Sauvegarder la séquence segmentée
   (f) Décrire et analyser/critiquer vos observations

3 Application - relevez le défi !

On souhaite dénombrer automatiquement le nombre de bandes blanches composant le passage piéton dans l'image passagepieton.jpg (FIG.1), à récupérer sur Moodle. Madame Thomas a obtenu le résultat en FIG.2. Ferez-vous aussi bien qu'elle ou même mieux ? !

FIGURE 1 – Image d'un passage piéton

1. Proposer sur le papier une chaine de traitement sous forme d'organigramme permettant de satisfaire l'objectif
2. L'implémenter en vous appuyant sur les méthodes vues en cours et les codes développés jusqu'à présent (TD2, TD3 et TD4)
3. Analyser/Critiquer vos résultats

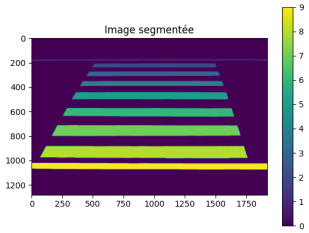

FIGURE 2 – Image segmentée